

Chapter 6 explain

شرح السلايدات : (1 - 17)

CHAPTER 6 – Basic SQL

الفصل 6 – SQL الأساسي

هذه هي بداية الفصل الذي يُقدّم لغة SQL، وهي لغة أساسية في قواعد البيانات العلائقية.

Chapter 6 Outline

مخطط الفصل 6

- SQL Data Definition and Data Types

تعريف البيانات وأنواع البيانات في SQL

- Specifying Constraints in SQL

تحديد القيود (Constraints) في SQL

- Basic Retrieval Queries in SQL

استعلامات الاسترجاع الأساسية في SQL

- INSERT, DELETE, and UPDATE Statements in SQL

أوامر الإدراج (INSERT)، والحذف (DELETE)، والتحديث (UPDATE) في SQL

✓ **ملاحظة مهمة:** الفصل يغطي الجوانب الأساسية لـ SQL، بدءًا من تعريف الجداول وانتهاءً بتعديل البيانات.

Basic SQL

الأساسي SQL

- SQL language

لغة SQL

- Considered one of the major reasons for the commercial success of relational databases

تُعدّ واحدة من الأسباب الرئيسية للنجاح التجاري لقواعد البيانات العلائقية

✓ **ملاحظة:** بدون SQL، لما كانت قواعد البيانات العلائقية شائعة بهذا الشكل. فهي لغة معيارية وسهلة الاستخدام نسبيًا.

- SQL
SQL
- The origin of SQL is relational predicate calculus called tuple calculus (see Ch.8) which was proposed initially as the language SQUARE
تعود أصول SQL إلى حساب التعبير العلائقي (predicate calculus) المعروف بـ "tuple calculus" انظر الفصل (8)، والذي تم اقتراحه في البداية كلغة تُسمى SQUARE

✓ **ملاحظة:** SQL ليست لغة برمجة تقليدية، بل لغة استعلام مبنية على أسس رياضية (منطق رياضي).

- SQL Actually comes from the word "SEQUEL" which was the original term used in the paper: "SEQUEL TO SQUARE" by Chamberlin and Boyce
في الواقع، أنت كلمة SQL من "SEQUEL"، وهي الاسم الأصلي المستخدم في الورقة البحثية: "SEQUEL TO SQUARE" من قبل Chamberlin و Boyce
- IBM could not copyright that term, so they abbreviated to SQL and copyrighted the term SQL
لم تتمكن IBM من تسجيل حقوق الملكية (Copyright) على كلمة "SEQUEL"، لذا اختصرتها إلى SQL وسجلت حقوقها على الاسم SQL
- Now popularly known as "Structured Query Language"
تُعرف الآن شعبياً باسم "لغة الاستعلام البنوية" أو "لغة الاستعلام المنظمة"

✓ **ملاحظة:** الاسم الرسمي الآن هو **SQL**، وتنطق أحياناً "سيكويل" (SEQUEL) أو حرفياً "إس كيو إل" (S-Q-L).

- SQL is an informal or practical rendering of the relational data model with syntax (Syntax) هي تجسيد غير رسمي أو عملي للنموذج العلائقي، مزوداً بقواعد نحوية SQL

✓ **ملاحظة:** SQL تُطبّق مفاهيم النموذج العلائقي (مثل الجداول، الصفوف، الأعمدة) بطريقة عملية وسهلة للاستخدام.

SQL Data Definition, Data Types, Standards

تعريف البيانات، وأنواع البيانات، ومعايير SQL

- Terminology:

المصطلحات:

- Table, row, and column used for relational model terms relation, tuple, and attribute
تُستخدم مصطلحات "جدول، صف، عمود" بدلاً من المصطلحات النظرية "علاقة، صف، صفة" في النموذج العلائقي

✓ **ملاحظة:** هذا التبسيط في المصطلحات يجعل SQL أكثر سهولة للمستخدمين.

- CREATE statement

أمر CREATE

- Main SQL command for data definition
لتعريف البيانات SQL الأمر الرئيسي في

✓ **ملاحظة:** يتم استخدام CREATE لإنشاء الجداول، والقيود، والمؤشرات (Indexes)، وغيرها.

- The language has features for:

تتضمن اللغة ميزات لـ:

- Data definition (تعريف البيانات)
- Data Manipulation (معالجة البيانات)
- Transaction control (التحكم في المعاملات – انظر الفصل 20)
- Indexing (الفهرسة – الفصل 17)
- Security specification (الفصل 30 – REVOKE و GRANT: تحديد الأمان)
- Active databases (قواعد البيانات النشطة – الفصل 26)
- Multi-media (الوسائط المتعددة – الفصل 26)
- Distributed databases (الوسائط المتعددة – الفصل 23) وغيرها (قواعد البيانات الموزعة – الفصل 23)

✓ **ملاحظة مهمة:** SQL لغة شاملة، لا تقتصر فقط على استعلامات، بل تشمل إدارة الأمان، المعاملات، التوزيع، وحتى الوسائط المتعددة.

SQL Standards

معايير SQL

- SQL has gone through many standards: starting with SQL-86 or SQL 1.A
مرت SQL بالعديد من المعايير، بدءًا من SQL-86 أو SQL 1.A
- SQL-92 is referred to as SQL-2
يُشار إلى SQL-92 باسم SQL-2

✓ **ملاحظة:** SQL-92 هو معيار مهم جدًا، وما زال يُستخدم كمرجع في العديد من الأنظمة.

- Later standards (from SQL-1999) are divided into core specification and specialized extensions
المعايير اللاحقة (من SQL-1999 فصاعدًا) تُقسّم إلى مواصفات أساسية (Core) وامتدادات متخصصة (Extensions)
- The extensions are implemented for different applications – such as data mining, data warehousing, multimedia etc.
تُطبّق هذه الامتدادات في تطبيقات مختلفة مثل تنقيب البيانات، مستودعات البيانات، الوسائط المتعددة، إلخ
- SQL-2006 added XML features (Ch. 13)
أضاف SQL-2006 ميزات XML (انظر الفصل 13)
- In 2008 they added Object-oriented features (Ch. 12)
في 2008، أُضيفت ميزات كائنية التوجه (OOP) (انظر الفصل 12)
- SQL-3 is the current standard which started with SQL-1999. It is not fully implemented in any RDBMS
وهو غير مُطبّق بالكامل في أي نظام لإدارة قواعد البيانات العلائقية. SQL-1999 هو المعيار الحالي الذي بدأ مع SQL-3 (RDBMS)

✓ **ملاحظة مهمة جدًا:** لا يوجد نظام قواعد بيانات يدعم كل ميزات SQL-3 بالكامل. كل شركة (مثل Oracle، MySQL، SQL Server) تُطبّق جزءًا من المعيار مع إضافات خاصة.

Schema and Catalog Concepts in SQL

مفاهيم المخطط (Schema) والفهرس (Catalog) في SQL

- We cover the basic standard SQL syntax – there are variations in existing RDBMS systems

الحالية RDBMS القياسية الأساسية – توجد اختلافات في أنظمة SQL نغطي بنية

✓ ملاحظة: تختلف كتابة الأوامر قليلاً بين Oracle و MySQL و SQL Server، لكن القواعد الأساسية واحدة.

- SQL schema

المخطط (Schema) في SQL

- Identified by a schema name

يُعرف باسم مخطط

- Includes an authorization identifier and descriptors for each element

يحتوي على معرف تفويض (Authorization ID) ومواصفات لكل عنصر

- Schema elements include

عناصر المخطط تشمل:

- Tables (الجدول)
- Constraints (القيود)
- Views (العروض)
- Domains (النطاقات)
- And other constructs (وغيرها من العناصر)

✓ ملاحظة: المخطط هو "حاوية" تنظم الكائنات في قاعدة البيانات.

- Each statement in SQL ends with a semicolon

(;) ينتهي بفاصلة منقوطة SQL كل أمر في

✓ ملاحظة ضرورية: في بعض الأنظمة (مثل MySQL)، يمكن تجاهل الفاصلة المنقوطة في سطر واحد، لكنها مطلوبة عند تنفيذ أكثر من أمر.

Schema and Catalog Concepts in SQL (cont'd.)

مفاهيم المخطط والفهرس في SQL (تابع)

- CREATE SCHEMA statement

أمر CREATE SCHEMA

- Example:

```
CREATE SCHEMA COMPANY AUTHORIZATION 'Jsmith';
```

✓ شرح المثال:

- يُنشئ مخططًا باسم COMPANY
- المستخدم Jsmith هو المالك (المُفَوِّض) لهذا المخطط
- بعد إنشاء المخطط، يمكن إنشاء جداول وقيود بداخله

⚠ ملاحظة: ليس كل نظام دعم يدعم CREATE SCHEMA بشكل مباشر (مثل بعض إصدارات MySQL القديمة)، لكنه موجود في Oracle و SQL Server.

- Catalog

الفهرس (Catalog)

- Named collection of schemas in an SQL environment
مجموعة مسماة من المخططات (Schemas) في بيئة SQL
- SQL also has the concept of a cluster of catalogs
تحتوي SQL أيضًا على مفهوم "تجمع من الفهارس" (Cluster of Catalogs)

✓ ملاحظة:

- Schemas يحتوي على عدة > Catalog
- Tables, Views, Constraints... يحتوي على عدة > Schema

مثلاً:

- Catalog: University_DB
- Schemas: Students , Courses , HR

- كل Schema يحتوي جداوله الخاصة

The CREATE TABLE Command in SQL

أمر CREATE TABLE في SQL

- Specifying a new relation

تحديد علاقة جديدة

- Provide name of table

تحديد اسم الجدول

- Specify attributes, their types, and initial constraints

تحديد السمات (الأعمدة)، وأنواعها، والقيود الأولية

- Can optionally specify schema:

يمكن اختياريًا تحديد المخطط:

- CREATE TABLE COMPANY.EMPLOYEE...

→ الجدول EMPLOYEE في المخطط COMPANY

- CREATE TABLE EMPLOYEE...

→ الجدول في المخطط الافتراضي (Default Schema)

The CREATE TABLE Command in SQL (cont'd.)

أمر CREATE TABLE في SQL (تابع)

- Base tables (base relations)

الجدول الأساسية (العلاقات الأساسية)

- Relation and its tuples are actually created and stored as a file by the DBMS
(DBMS) العلاقة وصفوها تنشأ فعلياً وتُخزن كملف بواسطة نظام إدارة قاعدة البيانات

✓ ملاحظة: الجداول الأساسية تُخزن على القرص، ويمكن تنفيذ عمليات عليها (SELECT, INSERT, ...)

- Virtual relations (views)

العلاقات الافتراضية (العروض)

- Created through the CREATE VIEW statement

تُنشأ باستخدام أمر CREATE VIEW

- Do not correspond to any physical file

لا تتوافق مع أي ملف فيزيائي

✓ ملاحظة: العرض (View) هو استعلام محفوظ، يُعامل كجدول، لكنه لا يحتوي بيانات حقيقية، بل يُعيد نتائجه من الجداول الأساسية عند الاستخدام.

COMPANY relational database schema (Fig. 5.7)

مخطط قاعدة بيانات COMPANY العلائقية (الشكل 5.7)

⚠ هذه شريحة تحتوي صورة لمخطط قاعدة البيانات (ER Diagram أو Relational Schema)، لكنها غير متضمنة في النص.
ومع ذلك، من السياق، نعرف أن:

- قاعدة بيانات COMPANY تحتوي على جداول مثل:

- EMPLOYEE
- DEPARTMENT
- DEPT_LOCATIONS
- PROJECT
- WORKS_ON
- DEPENDENT

✓ ملاحظة: هذا المخطط سيكون مرجعاً لبقية الأمثلة في الفصل.

One possible database state for the COMPANY relational database schema (Fig. 5.6)

حالة ممكنة لقاعدة بيانات COMPANY (الشكل 5.6)

هذه شريحة تحتوي على بيانات مثال (Sample Data) في الجداول.
تُظهر:

- بيانات الموظفين في جدول EMPLOYEE
- بيانات الإدارات في DEPARTMENT
- وغيرها

✓ ملاحظة: هذه البيانات تُستخدم لاحقًا في أمثلة الاستعلامات (Queries) لعرض النتائج.

One possible database state for the COMPANY relational database schema – continued (Fig. 5.6)

حالة قاعدة البيانات – استمرار (الشكل 5.6)

استمرار للبيانات في الجداول الأخرى مثل:

- WORKS_ON
- DEPENDENT
- PROJECT

✓ ملاحظة: من المهم فهم هيكل البيانات لفهم نتائج الاستعلامات لاحقًا.

SQL CREATE TABLE data definition statements for defining the COMPANY schema from Figure 5.7 (Fig. 6.1)

أوامر CREATE TABLE لتعريف مخطط COMPANY من الشكل 5.7 (الشكل 6.1)

هذه الشريحة (والتالية) تُظهر أوامر SQL الحقيقية لإنشاء الجداول.

المثال من الشريحة:

```
CREATE TABLE EMPLOYEE (  
  Fname VARCHAR(15) NOT NULL,  
  Minit CHAR,
```

```
Lname VARCHAR(15) NOT NULL,
Ssn CHAR(9) NOT NULL,
Bdate DATE,
Address VARCHAR(30),
Sex CHAR,
Salary DECIMAL(10,2),
Super_ssn CHAR(9),
Dno INT NOT NULL,
PRIMARY KEY (Ssn),
FOREIGN KEY (Super_ssn) REFERENCES EMPLOYEE(Ssn),
FOREIGN KEY (Dno) REFERENCES DEPARTMENT(Dnumber)
);
```

تحليل المثال: ✓

- VARCHAR(15): نص متغير الطول، بحد أقصى 15 حرفاً
- CHAR: حرف واحد (أو طول افتراضي)
- CHAR(9): نص ثابت الطول (مثل رقم الضمان الاجتماعي)
- DATE: تاريخ بالصيغة YYYY-MM-DD
- DECIMAL(10,2): عدد عشري، 10 أرقام إجمالاً، منها 2 بعد الفاصلة
- NOT NULL: لا يُسمح بالقيمة الفارغة (NULL)
- PRIMARY KEY (Ssn): SSN المفتاح الأساسي هو
- FOREIGN KEY ... REFERENCES: مفتاح أجنبي يشير إلى جدول آخر

ملاحظة مهمة: ✓

- يجب تعريف الجداول بترتيب منطقي (مثلاً: يجب تعريف DEPARTMENT قبل EMPLOYEE إذا كان EMPLOYEE يشير إليه)
- أو استخدام ALTER TABLE لاحقاً لإضافة القيود

Attribute Data Types and Domains in SQL

أنواع بيانات السمات ونطاقات القيم في SQL

Basic data types

الأنواع الأساسية للبيانات

• Numeric data types

أنواع البيانات العددية

- Integer numbers:

الأعداد الصحيحة:

- INTEGER, INT, SMALLINT

- Floating-point (real) numbers:

الأعداد العشرية (الحقيقية):

- FLOAT أو REAL
- DOUBLE PRECISION

✓ ملاحظة:

- INT : 32-bit عادةً
- SMALLINT : 16-bit
- DOUBLE PRECISION : دقة عالية جدًا

- **Character-string data types**

أنواع بيانات النصوص

- Fixed length:

طول ثابت:

- CHAR(n)
- CHARACTER(n)

- Varying length:

طول متغير:

- VARCHAR(n)
- CHAR VARYING(n)
- CHARACTER VARYING(n)

✓ ملاحظة:

- CHAR(10) يخصص دائمًا 10 خانات، حتى لو كان النص أقصر (يُملأ بمسافات)
- VARCHAR(10) يستخدم فقط ما يحتاجه + 2 بايت إضافية للطول

Attribute Data Types and Domains in SQL (cont'd.)

أنواع بيانات السمات ونطاقات القيم في SQL (تابع)

- **Bit-string data types**

أنواع بيانات السلاسل الثنائية

- Fixed length: BIT(n)
- Varying length: BIT VARYING(n)

✓ ملاحظة: تُستخدم لتخزين بيانات ثنائية (مثل: 10101011)

- **Boolean data type**

نوع البيانات المنطقي

- Values of: TRUE or FALSE or NULL
القيم: صحيح أو خاطئ أو فارغ

✓ ملاحظة: في بعض الأنظمة (مثل MySQL)، يُستخدم TINYINT ك Boolean.

• DATE data type

نوع البيانات التاريخية

- Ten positions

10 مواضع

- Components are YEAR, MONTH, and DAY in the form YYYY-MM-DD

تتكون من السنة، الشهر، اليوم بالشكل: YYYY-MM-DD

- Multiple mapping functions available in RDBMSs to change date formats

توجد دالات تعيين متعددة في أنظمة قواعد البيانات لتغيير تنسيق التاريخ

✓ ملاحظة:

- التاريخ يُخزن داخليًا كرقم، ويُعرض بالشكل YYYY-MM-DD

- أمثلة: 05-04-2025 , 25-12-1990

✓ خلاصة شاملة للملاحظات النهائية (Exam Notes)

الملاحظة	الموضوع
tuple calculus وتعتمد على SEQUEL، نشأت من	SQL
SQL-86 → SQL-92 (SQL2) → SQL-1999 (SQL3) → SQL-2006 (XML) → SQL-2008 (OOP)	المعايير
غير مُطبق بالكامل في أي نظام	SQL-3
حاوية للجدول، القيود، العروض... ويملكه مستخدم (Authorization)	المخطط (Schema)
مجموعة من المخططات	الفهرس (Catalog)
يُستخدم لتعريف الجداول الأساسية (مخزنة فعليًا)	CREATE TABLE
يُستخدم للعروض (غير مخزنة فعليًا)	CREATE VIEW
CHAR vs VARCHAR – INT – DATE – BOOLEAN	الأنواع
كل أمر SQL ينتهي بـ ;	الفاصلة المنقوطة
يجب تعريف الجداول بترتيب منطقي (المفتاح المرجعي أولاً أو استخدام ALTER لاحقًا)	الترتيب

Slide 6-19: Attribute Data Types and Domains in SQL (cont'd.)

أنواع بيانات السمات والنطاقات في SQL (تابع)

Additional data types

أنواع بيانات إضافية

1. Timestamp data type

نوع البيانات الزمني (Timestamp)

- يحتوي على حقل **DATE** وحقل **TIME** معًا.
- يحتوي على **6** خانات على الأقل للجزء العشري من الثانية (أي دقة تصل إلى الميكروثانية).

مثال: 14:30:25.123456 05-10-2023

- يمكن إضافة مؤهل اختياري **WITH TIME ZONE** لتحديد المنطقة الزمنية.

مثال: **TIMESTAMP WITH TIME ZONE** يُستخدم عند الحاجة لمتابعة الوقت عبر مناطق زمنية مختلفة (مثل التطبيقات العالمية).

- ملاحظة: يُستخدم لتسجيل الأحداث الزمنية بدقة عالية مثل السجلات (Logs) أو المعاملات.

2. INTERVAL data type

نوع البيانات الفاصل الزمني (INTERVAL)

- يُستخدم لتمثيل قيمة زمنية نسبية (مثل: 5 أيام، 2 ساعة و30 دقيقة).
- يُستخدم لزيادة أو تقليل قيمة مطلقة من التاريخ أو الوقت أو الطابع الزمني.

مثال: **INTERVAL '5' DAY** أو **INTERVAL '2:30' HOUR TO MINUTE**

- أنواع **INTERVAL**:

- **YEAR TO MONTH**
- **DAY TO SECOND**

- يُستخدم غالبًا مع عمليات حسابية على التواريخ:

```
SELECT CURRENT_TIMESTAMP + INTERVAL '1' DAY;
```

3. التحويل إلى النص (String Conversion)

- يمكن تحويل أنواع **DATE** , **TIME** , **TIMESTAMP** , و **INTERVAL** إلى صيغ نصية (string formats) لتسهيل المقارنة.
- يتم ذلك باستخدام دالة **CAST** أو **TO_CHAR** (حسب نظام قاعدة البيانات).

```
CAST(birth_date AS VARCHAR(20))
```

- ملاحظة: هذا مفيد عند المقارنة أو عرض البيانات بالشكل المطلوب.

Slide 6-20: Attribute Data Types and Domains in SQL (cont'd.)

أنواع بيانات السمات والنطاقات في SQL (تابع)

Domain (النطاق)

- النطاق (Domain) هو اسم يُعطى لتعريف نوع بيانات معين، ثم يُستخدم في تعريف السمات.
- الفوائد:
- 1. سهولة التعديل: إذا تم استخدام نفس النطاق في عدة جداول، يمكن تغيير تعريفه مرة واحدة.
- 2. تحسين قابلية القراءة: يجعل الجدول أكثر وضوحًا.
- 3. التنظيم: يساعد في توحيد أنواع البيانات عبر قاعدة البيانات.
- مثال عملي:

```
;CREATE DOMAIN SSN_TYPE AS CHAR(9)
```

ثم نستخدمه في تعريف جدول:

```
) CREATE TABLE EMPLOYEE  
, Ssn SSN_TYPE PRIMARY KEY  
...  
;
```

إذا أردنا لاحقًا تغيير CHAR(9) إلى VARCHAR(11)، نُعدّل النطاق مرة واحدة فقط.

UDTs - أنواع معرفة من قبل المستخدم (TYPE)

- User Defined Types (UDTs) (Object-Oriented) تُستخدم في التطبيقات الكائنية.
- تُعرّف باستخدام الأمر:

```
CREATE TYPE
```

- تُستخدم عندما نحتاج أنواع بيانات معقدة (مثل: ADDRESS, PHONE_NUMBER ككائن كامل).
- ملاحظة: سيتم تغطيتها بالتفصيل في الفصل 12 (النماذج الكائنية).

Slide 6-21: Specifying Constraints in SQL

تحديد القيود في SQL

القيود الأساسية في النموذج العلاقي (Basic Constraints)

يوجد 3 أنواع أساسية من القيود يدعمها SQL:

1. Key Constraint (قيود المفتاح)

- لا يمكن أن تتكرر قيمة المفتاح الأساسي (Primary Key).
- يضمن التمييز بين السجلات.

مثال: لا يمكن أن يكون هناك موظفان لهما نفس Ssn.

2. Entity Integrity Constraint (قيود سلامة الكيان)

- لا يمكن أن تكون قيمة المفتاح الأساسي NULL.
- أي أن المفتاح الأساسي يجب أن يكون له قيمة.

مثال: Ssn لا يمكن أن يكون فارغًا.

3. Referential Integrity Constraint (قيود السلامة المرجعية)

- تُطبق على المفتاح الخارجي (Foreign Key).
- يجب أن تكون قيمة المفتاح الخارجي:
- إما مطابقة لقيمة المفتاح الأساسي في الجدول المرجعي.
- أو تكون NULL (إذا كان مسموحًا بذلك).

مثال: Dno في جدول EMPLOYEE يجب أن يشير إلى Dnumber موجود في DEPARTMENT.

Slide 6-22: Specifying Attribute Constraints

تحديد قيود السمات

قيود إضافية على نطاقات السمات

1. Default Value (القيمة الافتراضية)

- تُستخدم لتحديد قيمة تُدخل تلقائيًا إذا لم تُدخل قيمة يدويًا.
- تُستخدم بالكلمة المفتاحية: <DEFAULT <value

Salary DECIMAL(10,2) DEFAULT 30000

- ملاحظة: مفيدة في الحقول التي لها قيمة شائعة (مثل: الراتب الأساسي، الدولة الافتراضية).

2. NOT NULL

- تمنع إدخال القيمة NULL في السمة.

Dname VARCHAR(15) NOT NULL

- تُستخدم مع السمات التي يجب أن تحتوي على قيمة.

3. CHECK Clause (شرط التحقق)

- يُستخدم لفرض قيود على قيمة السمة.
- مثال:

```
;Dnumber INT NOT NULL CHECK (Dnumber > 0 AND Dnumber < 21)
```

- يضمن أن رقم القسم بين 1 و 20 فقط.
- يمكن استخدام تعبيرات منطقية (AND , OR , NOT).
- ملاحظة: لا يمكن استخدام CHECK للوصول إلى جداول أخرى (محدود بالصف الحالي فقط).

Slide 6-23: Specifying Key and Referential Integrity Constraints

تحديد قيود المفاتيح والمرجعية

1. PRIMARY KEY Clause

- يُستخدم لتحديد السمة (أو المجموعة) التي تكون المفتاح الأساسي.

```
Dnumber INT PRIMARY KEY
```

- يجمع بين NOT NULL و UNIQUE تلقائيًا.
- يمكن أن يكون مركبًا (مكوّنًا من أكثر من سمة):

```
PRIMARY KEY (Dnumber, Dname)
```

2. UNIQUE Clause

- يُستخدم لتحديد مفاتيح بديلة (Alternate Keys) أو ما يُعرف بـ المفاتيح المرشحة (Candidate Keys).

```
Dname VARCHAR(15) UNIQUE
```

- يسمح بوجود قيمة NULL مرة واحدة فقط (في بعض الأنظمة أكثر من مرة حسب التنفيذ).
- لا يفرض NOT NULL تلقائيًا (يجب كتابته يدويًا إذا أردت ذلك).
- ملاحظة: يمكن أن يكون هناك أكثر من UNIQUE في الجدول، لكن مفتاحًا أساسيًا واحدًا فقط.

Slide 6-24: Specifying Key and Referential Integrity Constraints (cont'd.)

تحديد قيود المفاتيح والمرجعية (تابع)

FOREIGN KEY Clause

- يُستخدم لربط جدول بآخر عبر مفتاح خارجي.
- الشكل العام:

الإجراءات المُفَعَّلة عند الانتهاك (Referential Triggered Actions)

عند حذف أو تحديث سجل في الجدول المرجعي، ماذا يحدث للسجلات المرتبطة؟

1. السلوك الافتراضي: REJECT (أو NO ACTION)

- يرفض العملية إذا كانت هناك سجلات مرتبطة.

مثال: لا يمكن حذف قسم له موظفون.

2. خيارات إضافية (ON DELETE / ON UPDATE):

• SET NULL :

- عند حذف/تحديث السجل المرجعي، تُجعل قيمة المفتاح الخارجي NULL.
- يجب أن يكون الحقل مسموحًا له بـ NULL .

• SET DEFAULT :

- تُعيّن القيمة إلى القيمة الافتراضية المحددة.

• CASCADE :

- يُحذف/يُحدّث السجل المرتبط تلقائيًا.
- مثال: عند حذف قسم، تُحذف جميع الأقسام الفرعية التابعة له.
- مناسب للعلاقات الهرمية أو العلاقات القوية (مثل: "Relationship" relations).

ملاحظة: تُكتب بهذه الصيغة:

```
FOREIGN KEY (Dno) REFERENCES DEPARTMENT(Dnumber)
ON DELETE CASCADE ON UPDATE SET NULL
```

Slide 6-25: Giving Names to Constraints

إعطاء أسماء للقيود

- يمكن تسمية القيود باستخدام الكلمة المفتاحية CONSTRAINT .
- السبب: تسهيل تعديل أو حذف القيود لاحقًا باستخدام ALTER TABLE .
- الشكل العام:

```
CONSTRAINT constraint_name CHECK (Dnumber > 0)
```

- مثال كامل:

```
) CREATE TABLE DEPARTMENT
, Dname VARCHAR(15) NOT NULL
, Dnumber INT NOT NULL
, CONSTRAINT pk_dept PRIMARY KEY (Dnumber)
```

```
CONSTRAINT chk_dnum CHECK (Dnumber > 0 AND Dnumber < 21)
; (
```

- ملاحظة: بدون تسمية، يُؤد النظام اسمًا تلقائيًا (صعب التنبؤ به)، لذا يُفضل التسمية.

Slide 6-26: Default attribute values and referential integrity triggered action specification (Fig. 6.2)

القيم الافتراضية للسمات وتحديد إجراءات السلامة المرجعية (الشكل 6.2)

- هذه الشريحة تشير إلى رسم توضيحي (Fig. 6.2) يوضح:
 - كيفية استخدام DEFAULT في تعريف السمات.
 - كيفية تحديد إجراءات ON DELETE و ON UPDATE مع FOREIGN KEY.
 - بما أن الرسم غير متاح، نلخص:
 - يُظهر مثالًا عمليًا لكيفية تطبيق:
- على سمة DEFAULT 'UNKNOWN'.
- على مفتاح خارجي ON DELETE SET NULL.
- يُستخدم كمرجع بصري لكيفية كتابة الجملة الكاملة.

Slide 6-27: Specifying Constraints on Tuples Using CHECK

تحديد قيود على السجلات باستخدام CHECK

- يمكن وضع قيود على مستوى السجل (tuple) وليس فقط على السمة.
- تُستخدم جملة CHECK في نهاية CREATE TABLE.
- تُطبق على كل سجل على حدة.
- مثال:

```
) CREATE TABLE DEPARTMENT
...
, Dept_create_date DATE
, Mgr_start_date DATE
CHECK (Dept_create_date <= Mgr_start_date)
; (
```

يضمن أن مدير القسم لا يمكن أن يبدأ قبل إنشاء القسم.

- ملاحظات:
- يمكن أن يشترك أكثر من سمة في نفس CHECK.
- لا يمكن الرجوع إلى جداول أخرى (محدود بالسجل الحالي فقط).

- يُستخدم لفرض قواعد أعمال (Business Rules) على مستوى السجل.

Slide 6-28: Basic Retrieval Queries in SQL

استعلامات الاسترجاع الأساسية في SQL

- **SELECT statement** هي الجملة الأساسية لاسترجاع البيانات.
- ملاحظة مهمة:
- يسمح بتكرار السجلات (أي يمكن أن يكون هناك سجلان متماثلان تمامًا) SQL.
- هذا يختلف عن النموذج العلاقي الرياضي الذي يعتمد على النماذج المجموعة (Set Theory) حيث لا يُسمح بالتكرار.
- لذلك، يُقال إن SQL يتبع سلوك **Multiset** أو **Bag** (مجموعة غير مميزة).
- الحلقة: يمكن استخدام **DISTINCT** لإزالة التكرار:

... **SELECT DISTINCT**

- ملاحظة: قد تستخدم بعض الأنظمة **tuple-id** داخليًا لتمييز السجلات المتماثلة.

Slide 6-29: The SELECT-FROM-WHERE Structure of Basic SQL Queries

هيكل استعلامات SQL الأساسية: SELECT-FROM-WHERE

- الشكل الأساسي لاستعلام SQL:

```
<SELECT <attribute_list  
<FROM <table_list  
<WHERE <condition
```

- وظيفة كل جزء:
- **SELECT** (الإسقاط - Projection) يحدد أية أعمدة (سمات) نريد استرجاعها.
- **FROM** يحدد الجداول التي نستعلم منها.
- **WHERE** (الاختيار - Selection) يحدد الشرط لتصفية السجلات.

Slide 6-30: The SELECT-FROM-WHERE Structure of Basic SQL Queries (cont'd.)

هيكل استعلامات SQL الأساسية (تابع)

Logical Comparison Operators

- = , > , < , <= , >= (أو != في بعض الأنظمة)
- تُستخدم في شرط WHERE .

Projection Attributes

سمات الإسقاط

- السمات المذكورة في SELECT هي التي تظهر في النتيجة.
- يمكن استخدام * لاسترجاع جميع السمات:

```
;SELECT * FROM EMPLOYEE
```

Selection Condition

شرط الاختيار

- شرط منطقي يجب أن يكون صحيحًا لاسترجاع السجل.
- قد يحتوي على شروط ارتباط (Join Conditions) عندما تُستعلم أكثر من جدول واحد (تُغطى لاحقًا في الفصل 8).

Slides 6-31 & 6-32: Basic Retrieval Queries

استعلامات الاسترجاع الأساسية

- هاتان الشريحتان تحتويان على أمثلة تطبيقية لاستعلامات SELECT .
- بما أن المحتوى غير مفصل في النص، نفترض أنه يعرض أمثلة مثل:

```
;SELECT Fname, Lname FROM EMPLOYEE WHERE Dno = 5
```

استرجاع أسماء الموظفين في القسم 5.

- التركيز على:

- استخدام WHERE لتصفية البيانات.
- استخدام SELECT لاختيار الأعمدة.
- استخدام FROM لتحديد الجدول.

Slide 6-33: Ambiguous Attribute Names

أسماء السمات المبهمة

- يمكن أن تُستخدم نفس الاسم لسمتين في علاقتين مختلفتين.

مثال: Name في جدول EMPLOYEE وجدول DEPARTMENT.

- ولكن عند كتابة استعلام يتضمن كلا الجدولين، يصبح الاسم مبهماً.
- الحل: **qualification** (تأليف الاسم) باستخدام اسم الجدول:

```
SELECT EMPLOYEE.Name, DEPARTMENT.Name
FROM EMPLOYEE, DEPARTMENT
WHERE EMPLOYEE.Dno = DEPARTMENT.Dnumber
```

- بدون التأليف، يُعتبر الاستعلام خاطئاً.

Slide 6-34: Aliasing, and Renaming

الأسماء المستعارة وإعادة التسمية

- **Aliasing (الأسماء المستعارة)**

- يُستخدم لاختصار أسماء الجداول أو استخدام نفس الجدول مرتين في استعلام واحد.
- تُستخدم الكلمة AS (يمكن حذفها في معظم الأنظمة).
- مثال (الاستعلام 8):

```
SELECT E.Fname, E.Lname, S.Fname, S.Lname
FROM EMPLOYEE AS E, EMPLOYEE AS S
WHERE E.Super_ssn = S.Ssn
```

- E : يمثل الموظف.
- S : يمثل المدير (نفس الجدول).

- يُستخدم لاسترجاع اسم كل موظف واسمه مديره المباشر.

• توصية:

- يُفضل استخدام الأسماء المستعارة لتبسيط الاستعلامات الطويلة.
- يُستخدم البادئة عند وجود سمات متشابهة من جداول مختلفة.

Slide 6-35: Aliasing, Renaming and Tuple Variables (cont'd.)

الأسماء المستعارة وإعادة التسمية (تابع)

- يمكن أيضاً إعادة تسمية السمات عند استخدام AS :

```
EMPLOYEE AS E(Fn, Mi, Ln, Ssn, Bd, Addr, Sex, Sal, Sssn, Dno)
```

- هنا، تم تعيين أسماء مستعارة للسمات داخل الجدول.

- إلخ ، Fname بدلاً من Fn.

• ملاحظة:

- EMPLOYEE يشير إلى كل صف في (Tuple Variable) **Tuple Variable** يصبح متغير سجل E
- تُستخدم هذه الميزة في الاستعلامات المعقدة أو عند استخدام الجداول الظاهرية (Views).
- AS يمكن حذفها في معظم أنظمة SQL (مثل MySQL، PostgreSQL).

ملخص عام للملاحظات المهمة

المفهوم	الملاحظة
Timestamp	يحتوي على تاريخ، وقت، وجزء عشري للثانية، مع دعم للمنطقة الزمنية.
INTERVAL	لتمثيل فترات زمنية نسبية (مثل: +5 أيام).
Domain	يُستخدم لتوحيد نوع البيانات عبر الجداول.
CHECK	يُستخدم لقيود على السمات أو السجلات، لكن لا يصل إلى جداول أخرى.
FOREIGN KEY Actions	تُحدد السلوك عند الحذف أو التحديث CASCADE , SET NULL , SET DEFAULT
CONSTRAINT naming	ضروري لتعديل القيود لاحقاً.
SELECT	يسمح بالتكرار (Bag)، استخدم DISTINCT للإزالة.
Ambiguous names	يجب تأليفها باسم الجدول.
Aliasing	أساسي للاستعلامات المعقدة، خاصة مع الجداول الذاتية.

شرح السلايدات : (35 - 52)

Unspecified WHERE Clause and Use of the Asterisk

عدم تحديد شرط WHERE واستخدام النجمة (*)

Missing WHERE clause

WHERE عدم وجود شرط

- إذا لم يتم تضمين جملة WHERE في استعلام SELECT ، فهذا يعني أنه لا يوجد شرط على اختيار الصفوف (tuples).
- النتيجة: يتم اختيار جميع الصفوف الممكنة من الجداول المذكورة في جملة FROM .
- إذا كان هناك أكثر من جدول في FROM بدون شرط ربط (مثل WHERE E.Ssn = W.Essn)، فإن الناتج سيكون الضرب الديكارتي (Cartesian Product) بين الجداول.
- مثال: إذا كان الجدول الأول يحتوي على 10 صفوف، والثاني 5 صفوف، فسيكون الناتج 50 صفًا (10 × 5).
- هذه النتيجة غالبًا ما تكون غير مرغوبة، لذا يجب الانتباه لوجود شرط ربط مناسب في WHERE .

♦ ملاحظة مهمة: عدم وجود WHERE لا يعني "لا نتائج"، بل يعني "كل النتائج الممكنة".

Unspecified WHERE Clause and Use of the Asterisk (cont'd.)

عدم تحديد شرط WHERE واستخدام النجمة (*) (تابع)

- Specify an asterisk (*)

(*) استخدام النجمة

- عند استخدام * في جملة SELECT ، فإنها تعني: استرجاع جميع القيم (الأعمدة) للصفوف المختارة.
- يمكن كتابة النجمة مع اسم الجدول للتحديد الدقيق:

مثال: `SELECT EMPLOYEE.* FROM EMPLOYEE` ;

- هذا يُستخدم عندما يكون هناك أكثر من جدول في FROM ، ونريد فقط أعمدة جدول معين.

📌 ملاحظة: SELECT * يُستخدم لسهولة الكتابة، لكنه غير مستحسن في التطبيقات الكبيرة لأنه:

- يُبطئ الأداء (يُحمل بيانات غير ضرورية).
- يجعل الكود أقل وضوحًا.

Tables as Sets in SQL

الجدول كمجموعة في SQL

- SQL does not automatically eliminate duplicate tuples in query results

لا يُزيل SQL التكرارات تلقائيًا من نتائج الاستعلامات

- على عكس نظرية المجموعات، فإن SQL يسمح بالتكرار في نتائج الاستعلامات.
- مثال: إذا كان هناك صفان متماثلان تمامًا في الناتج، فسيظهران كصفتين منفصلتين.

- Use the keyword DISTINCT in the SELECT clause

استخدام الكلمة المفتاحية DISTINCT في جملة SELECT

- لحذف التكرارات، نستخدم DISTINCT :

`SELECT DISTINCT Salary FROM EMPLOYEE` ;

- الناتج: كل قيمة راتب تظهر مرة واحدة فقط، حتى لو كان هناك 10 موظفين بنفس الراتب.

📌 ملاحظة مهمة:

- SELECT. تُطبق على جميع الأعمدة المحددة في DISTINCT.

- مثال: `SELECT DISTINCT Fname, Lname` تعني أن التركيبة (الاسم الأول + الاسم الأخير) يجب أن تكون فريدة.

- لا تُستخدم DISTINCT مع الدوال التجميعية مثل COUNT إلا إذا أردت عدد القيم المختلفة.

Tables as Sets in SQL (cont'd.)

الجدول كمجموعة في SQL (تابع)

- Set operations

عمليات المجموعات

العمليات المنطقية على المجموعات SQL تدعم

العملية	الوصف
UNION	يجمع النتائج من استعلامين مع حذف التكرارات.
EXCEPT	يُرجع الصفوف الموجودة في الاستعلام الأول وليس في الثاني (فرق المجموعات).
INTERSECT	يُرجع الصفوف المشتركة بين الاستعلامين.

- Multiset versions (مع الحفاظ على التكرارات)

- UNION ALL : يجمع النتائج بما في ذلك التكرارات.
- EXCEPT ALL : يحذف كل تكرار في الثاني من الأول حسب العدد.
- INTERSECT ALL : يُرجع التكرارات المشتركة بأقل عدد مكرر.

- Type compatibility is needed

ضرورة التوافق في الأنواع

- لكي تُستخدم هذه العمليات، يجب أن يكون:
 - عدد الأعمدة متساويًا في كلا الاستعلامين.
 - أنواع البيانات متوافقة (مطابقة أو قابلة للتحويل) في كل عمود متناظر.
 - الترتيب مهم: العمود الأول في الأول يقابله الأول في الثاني.

ملاحظة: 

- UNION هو الأكثر استخدامًا.
- (القديم MySQL مثل) غير مدعومين في بعض أنظمة قواعد البيانات INTERSECT و EXCEPT.

Substring Pattern Matching and Arithmetic Operators

مطابقة النمط الجزئي للنصوص والعمليات الحسابية

- LIKE comparison operator

LIKE عامل المقارنة

- يُستخدم لمطابقة نمط معين في سلسلة نصية (string).
- يُستخدم غالبًا مع WHERE .

الرمز	المعنى
%	يحل محل أي عدد من الأحرف (بما في ذلك الصفر).

الرمز	المعنى
_ (underscore)	يحل محل حرف واحد فقط.

• أمثلة:

```
; 'WHERE Address LIKE '%Houston,TX
```

• يُرجع كل العناوين التي تحتوي على "Houston,TX" في أي مكان.

```
; 'WHERE Ssn LIKE '__1__8901
```

• يبحث عن أرقام الضمان الاجتماعي حيث يكون الحرف الثالث هو 1، والحرف السابع والثامن 89، والتاسع 0، والعاشر 1.

• _ = حرف واحد، __ = حرفين، وهكذا.

◆ ملاحظة:

- في بعض الأنظمة، وغير حساس في أخرى (يعتمد على الإعدادات) (Case-sensitive) حساس للحالة LIKE
- يمكن استخدام NOT LIKE لعكس الشرط.

• BETWEEN comparison operator

عامل المقارنة BETWEEN

• يُستخدم للتحقق مما إذا كانت القيمة داخل نطاق مغلق (شامل الحدين).

```
WHERE Salary BETWEEN 30000 AND 40000;
```

• تعادل:

```
WHERE Salary >= 30000 AND Salary <= 40000;
```

• مثال من النص:

```
;WHERE (Salary BETWEEN 30000 AND 40000) AND Dno = 5
```

• يختار الموظفين في القسم 5 برتبة بين 30,000 و 40,000 شاملة.

◆ ملاحظة:

- يُستخدم مع الأرقام، التواريخ، والنصوص BETWEEN
- يمكن استخدام NOT BETWEEN لعكس الشرط.

Arithmetic Operations

العمليات الحسابية

- **Standard arithmetic operators**

العوامل الحسابية القياسية

SELECT : العمليات التالية داخل SQL تدعم

المعنى	العامل
الجمع	+
الطرح	-
الضرب	*
القسمة	/

- **Query 13 Example:**

عرض الراتب الجديد إذا حصل كل موظف يعمل في مشروع "ProductX" على زيادة 10%.

```
SELECT E.Fname, E.Lname, 1.1 * E.Salary AS Increased_sal
FROM EMPLOYEE AS E, WORKS_ON AS W, PROJECT AS P
WHERE E.Ssn = W.Essn
      AND W.Pno = P.Pnumber
      AND P.Pname = 'ProductX';
```

- $1.1 * E.Salary$: يحسب الراتب بعد الزيادة.
- AS Increased_sal : يُعطي العمود اسمًا مخصصًا في الناتج (Alias).

📌 ملاحظات مهمة:

- يمكن استخدام العمليات الحسابية مباشرة في SELECT .
- يمكن تسمية العمود الناتج بـ AS .
- العمليات لا تُغير القيمة في الجدول، بل تُعرض مؤقتًا في الناتج.

Ordering of Query Results

ترتيب نتائج الاستعلام

- **Use ORDER BY clause**

استخدام جملة ORDER BY

- تُستخدم لترتيب النتائج بناءً على عمود أو أكثر.

- **Keywords:**

- الترتيب التصاعدي (من الصغير إلى الكبير) — هو الافتراضي : ASC .
- الترتيب التنازلي (من الكبير إلى الصغير) : DESC .

• مثال:

```
ORDER BY D.Dname DESC, E.Lname ASC, E.Fname ASC
```

- يرتب أولاً حسب اسم القسم تنازليًا.

- ثم حسب الاسم الأخير تصاعديًا.
- ثم حسب الاسم الأول تصاعديًا.

ملاحظات:

- `ORDER BY` توضع دائمًا في نهاية الاستعلام.
- يمكن الترتيب حسب أعمدة غير موجودة في `SELECT` (طالما موجودة في الجداول المستخدمة).
- الترتيب يُطبق بالتسلسل: الأولوية للعمود الأول، ثم الثاني عند التساوي.

Basic SQL Retrieval Query Block

الكود الأساسي لاسترجاع البيانات في SQL

(الشريحة 43-6 تُظهر هيكل الاستعلام الأساسي، لكن النص غير متوفر، لكن من السياق نفهم أنه يشير إلى:)

الهيكل العام لاستعلام `SELECT`:

```
SELECT [DISTINCT] <attribute_list>
FROM <table_list>
[WHERE <condition>]
[ORDER BY <attribute_list> [ASC|DESC]];
```

ملاحظة: كل جزء (ما عدا `SELECT` و `FROM`) اختياري.

INSERT, DELETE, and UPDATE Statements in SQL

أوامر إدراج، حذف، وتحديث البيانات في SQL

- الثلاثة أوامر تُستخدم لتعديل قاعدة البيانات (DML – Data Manipulation Language).

الوظيفة	الأمر
إضافة صفوف جديدة إلى الجدول.	INSERT
حذف صفوف من الجدول.	DELETE
تعديل قيم في صفوف موجودة.	UPDATE

ملاحظة عامة:

- هذه الأوامر تُنفذ على جدول واحد في كل مرة (إلا إذا استخدم `CASCADE`).
- يجب احترام قيود النزاهة (Integrity Constraints) مثل المفاتيح الأساسية والخارجية.

INSERT

إدراج بيانات

- **وظيفته:** إضافة صف (أو أكثر) إلى جدول.
- القيم يجب أن تكون بنفس الترتيب المُعرف في CREATE TABLE.
- تُفرض قيود النوع (data type) والنزاهة تلقائيًا.
- الشكل البسيط:

```
INSERT INTO EMPLOYEE VALUES ('John', 'Smith', '123456789', '1965-01-09',  
                                ;...)
```

- يجب تطابق عدد القيم ونوعها مع تعريف الجدول.
- إدراج متعدد (Bulk Insert) باستخدام استعلام:

```
INSERT INTO D5EMPS  
;SELECT E.* FROM EMPLOYEE E WHERE E.Dno = 5
```

- يُستخدم لنقل بيانات من جدول إلى آخر.

BULK LOADING OF TABLES

تحميل كميات كبيرة من البيانات

- إنشاء جدول جديد وتحميله ببيانات من جدول آخر:

```
CREATE TABLE D5EMPS LIKE EMPLOYEE  
(SELECT E.* FROM EMPLOYEE AS E WHERE E.Dno = 5)  
;WITH DATA
```

- LIKE EMPLOYEE : يُنشئ جدولًا بنفس هيكل الجدول .
- WITH DATA : يُحمّل البيانات فورًا .
- WITHOUT DATA : يُنشئ الجدول بدون بيانات (هيكل فقط).

💡 **ملاحظة:** هذه الصيغة قد تختلف حسب نظام قاعدة البيانات (مثل DB2 أو Oracle).

DELETE

حذف بيانات

- **وظيفته:** حذف صفوف من جدول.
- يتطلب WHERE لتحديد الصفوف المراد حذفها.
- إذا لم يُذكر WHERE ، فسيتم حذف جميع الصفوف من الجدول (يبقى الهيكل فقط).
- يُحترم النقاء المرجعي (Referential Integrity):

- لا يمكن حذف صف مُشار إليه من جدول آخر (ما لم يُستخدم ON DELETE CASCADE).

• مثال:

```
;DELETE FROM EMPLOYEE WHERE Dno = 5
```

- يحذف جميع الموظفين في القسم 5.

🔴 ملاحظات:

- DELETE إلى الصفر (Identity) لا يُعيد رقم التسلسل.
- يمكن التراجع عن الحذف باستخدام ROLLBACK إذا كان داخل معاملة (Transaction).

UPDATE

تحديث بيانات

- وظيفته: تعديل قيم في صفوف محددة.
- يتطلب:

- لتحديد الصفوف المستهدفة : WHERE
- لتحديد الأعمدة الجديدة والقيم : SET

• مثال:

```
UPDATE PROJECT
SET PLOCATION = 'Bellaire', DNUM = 5
;WHERE PNUMBER = 10
```

- يُغير موقع المشروع 10 إلى "Bellaire" ورقمه إلى 5.

UPDATE (contd.)

التحديث (تابع)

• مثال متقدم:

```
UPDATE EMPLOYEE
SET SALARY = SALARY * 1.1
;WHERE DNO IN (SELECT DNUMBER FROM DEPARTMENT WHERE DNAME = 'Research')
```

- يعطي موظفي قسم "Research" زيادة 10%.

• تفسير مهم:

- SALARY على اليمين = : تشير إلى القيمة القديمة
- SALARY على يسار = : تشير إلى القيمة الجديدة بعد التحديث

🔴 ملاحظات:

- يمكن استخدام استعلامات فرعية (Subqueries) في WHERE .
- التحديث قد يُؤثر على عدة صفوف.
- يجب التأكد من دقة شرط WHERE لتجنب التحديث الخاطئ.

ملخص عام ونصائح

المفهوم	ما يجب تذكره
WHERE مفقود	ناتج الضرب الديكارتي (Cartesian Product)
* في SELECT	استرجاع جميع الأعمدة
DISTINCT	إزالة التكرارات
_ مع % و LIKE	مطابقة نمطية للنصوص
BETWEEN	نطاق شامل (شامل الحدين)
العمليات الحسابية	تُستخدم مباشرة في SELECT
ORDER BY	يوضع في النهاية، ASC افتراضي
INSERT	إضافة صفوف، يجب احترام الترتيب
DELETE	حذف صفوف، WHERE مهم جدًا
UPDATE	تعديل قيم، SET و WHERE ضروريان
UNION , EXCEPT , INTERSECT	تتطلب توافق الأعمدة
WITH DATA	تُستخدم في إنشاء جدول مع تحميل البيانات

ترجمة للمفردات و الكلمات و الرموز

المصطلحات والكلمات الأساسية

الكلمة (إنجليزية)	الترجمة (عربية)	الشرح المبسط
SQL	إس كيو إل / لغة الاستعلام البنيوية	لغة تُستخدم لإدارة قواعد البيانات العلائقية (مثل إنشاء الجداول، استرجاع البيانات، التعديل عليها).
Structured Query Language	لغة الاستعلام البنيوية	الاسم الكامل لـ SQL، وتعني لغة منظمة لاسترجاع وتعديل البيانات.
Relational Database	قاعدة بيانات علائقية	نوع من قواعد البيانات يُنظم البيانات في جداول (علاقات) مرتبطة ببعضها.

الكلمة (إنجليزية)	الترجمة (عربية)	الشرح المبسط
Table	جدول	يعادل "العلاقة" في النموذج العلائقي. يحتوي على صفوف وأعمدة.
Row	صف	يعادل "الصيغة" (Tuple). يمثل سجلاً واحداً في الجدول.
Column	عمود	يعادل "السمة" (Attribute). يمثل حقلاً في الجدول (مثل: الاسم، الرقم).
Attribute	سمة / حقل	خاصية أو عمود في الجدول (مثل: Name, Age).
Tuple	صيغة	صف من البيانات في الجدول.
Relation	علاقة	اسم رياضي للجدول في النموذج العلائقي.
CREATE	إنشاء	أمر SQL لإنشاء كائنات قاعدة البيانات مثل الجداول أو المخططات.
Data Definition	تعريف البيانات	جزء من SQL يُستخدم لتحديد هيكل الجداول، الأعمدة، والقيود.
Data Manipulation	معالجة البيانات	جزء من SQL يُستخدم لاسترجاع أو إدراج أو تعديل أو حذف البيانات.
Data Type	نوع البيانات	يحدد نوع القيمة التي يمكن تخزينها في عمود (مثل رقم، نص، تاريخ).
Constraint	قيد / شرط	قاعدة تُفرض على البيانات للحفاظ على صحتها (مثل: لا تكرار في المفتاح الأساسي).
Schema	مخطط	هيكل قاعدة البيانات الذي يحتوي على الجداول، الأعمدة، القيود، إلخ.
Catalog	كتالوج	مجموعة من المخططات (Schemas) في بيئة قاعدة البيانات.
Authorization	صلاحية / تفويض	تحديد من له الحق في إنشاء أو تعديل المخطط.
Base Table	جدول أساسي	جدول مخزن فعلياً على القرص، يحتوي على بيانات حقيقية.
Virtual Relation (View)	علاقة افتراضية (عرض)	لا يُخزن فعلياً، بل هو نتيجة استعلام يمكن استدعاؤها لاحقاً.
CREATE VIEW	إنشاء عرض	أمر لإنشاء جدول افتراضي من نتيجة استعلام.

أنواع البيانات (Data Types) 1234

النوع (إنجليزية)	الترجمة	الشرح
INTEGER, INT	عدد صحيح	مثل: 1، 100، -5
SMALLINT	عدد صحيح صغير	نفس INT لكن بحجم أصغر (للتوفير في المساحة)
FLOAT, REAL	عدد عشري (عائم)	مثل: 2.5، 3.14

النوع (إنجليزية)	الترجمة	الشرح
DOUBLE PRECISION	عدد عشري دقيق	دقة أعلى من FLOAT
CHAR(n)	نص ثابت الطول	نص بطول محدد (n حرف)، مثل: CHAR(10) لاسم 10 أحرف بالضبط.
VARCHAR(n)	نص متغير الطول	نص يمكن أن يصل طوله إلى n حرف، لكن لا يشغل مساحة فارغة.
BIT(n)	سلسلة بت ثابتة	تُستخدم لتمثيل بيانات ثنائية (0 و 1) بعدد n بت.
BIT VARYING(n)	سلسلة بت متغيرة	مثل BIT لكن الطول غير ثابت.
BOOLEAN	منطقي	يأخذ إما TRUE (صحيح) أو FALSE (خاطئ) أو NULL (غير معرف).
DATE	تاريخ	بصيغة YYYY-MM-DD، مثل: 15-03-2024
TIME	وقت	بصيغة HH:MM:SS، مثل: 14:30:00
TIMESTAMP	طابع زمني	يجمع التاريخ والوقت، وغالباً يحتوي على أجزاء من الثانية.
INTERVAL	فترة زمنية	تمثل فرقاً بين توقيتين، مثل: +5 أيام، -2 ساعة.

القيد (Constraints)

المصطلح	الترجمة	الشرح
PRIMARY KEY	المفتاح الأساسي	عمود (أو مجموعة أعمدة) لا يمكن أن يتكرر ولا يمكن أن يكون فارغاً (NULL). يُستخدم لتمييز كل صف.
UNIQUE	فريد	يضمن أن القيم في العمود لا تتكرر (لكن يمكن أن تكون NULL مرة واحدة).
NOT NULL	غير فارغ	يمنع أن تكون القيمة فارغة في هذا العمود.
DEFAULT	افتراضي	قيمة تُستخدم تلقائياً إذا لم تُدخل قيمة عند الإدراج.
FOREIGN KEY	المفتاح الخارجي	عمود يشير إلى المفتاح الأساسي في جدول آخر، لربط الجداول.
Referential Integrity	السلامة المرجعية	تضمن أن القيمة في المفتاح الخارجي توجد فعلاً في الجدول المرجعي.
CHECK	تحقق	قيد يفرض شرط منطقي على العمود (مثل: العمر < 18).
CASCADE	تسلسل	إذا حُذف السجل المرجعي، يُحذف السجلات المرتبطة تلقائياً.
SET NULL	تعيين فارغ	إذا حُذف السجل المرجعي، تُجعل القيمة الخارجية فارغة.
SET DEFAULT	تعيين افتراضي	إذا حُذف السجل المرجعي، تُستخدم القيمة الافتراضية.
ON DELETE	عند الحذف	يحدد ما يحدث عند حذف السجل المرجعي.
ON UPDATE	عند التحديث	يحدد ما يحدث عند تعديل السجل المرجعي.

المصطلح	الترجمة	الشرح
CONSTRAINT	قيد	يمكن تسمية القيد (مثل: CONSTRAINT FK_EMP_DEPT) لسهولة التعديل لاحقاً.

استعلامات الاسترجاع (Retrieval Queries) 🔍

المصطلح	الترجمة	الشرح
SELECT	اختر / استرجع	يُستخدم لاسترجاع البيانات من الجدول.
FROM	من	يحدد الجدول (أو الجداول) التي تُسترجع منها البيانات.
WHERE	حيث	يحدد الشرط الذي يجب أن ينطبق على الصفوف المسترجعة.
Projection	إسقاط	اختيار الأعمدة المراد عرضها (مثل: SELECT name, age).
Selection	اختيار	اختيار الصفوف التي تحقق شرطاً معيناً (مثل: WHERE age > 30).
JOIN	اتحاد	دمج صفوف من جدولين بناءً على شرط (مثل: WHERE E.dept_id = D.id).
Ambiguous Attribute	سمة مبهمه	عندما يكون لعمود نفس الاسم في أكثر من جدول، يجب توضيح الجدول (مثل: EMPLOYEE.name).
Alias (AS)	اسم مستعار	اسم مؤقت لجدول أو عمود (مثل: EMPLOYEE AS E).
Tuple Variable	متغير صيغة	اسم يُستخدم للإشارة إلى صف معين في الجدول داخل الاستعلام.
CROSS PRODUCT	الضرب الديكارتي	نتيجة جدولين بدون شرط WHERE، ويُنتج جميع التوافيق الممكنة.
Asterisk (*)	نجمة	تعني "جميع الأعمدة"، مثل: SELECT * FROM EMPLOYEE .
DISTINCT	مميز	يزيل التكرارات من النتائج (مثل: SELECT DISTINCT city).
UNION	اتحاد	يجمع نتائج استعلامين، مع إزالة التكرارات.
EXCEPT	استثناء	يُرجع الصفوف الموجودة في الأول فقط وليس في الثاني.
INTERSECT	تقاطع	يُرجع الصفوف المشتركة بين الاستعلامين.
UNION ALL	اتحاد مع التكرار	مثل UNION لكن مع السماح بالتكرار.
LIKE	مثل	يُستخدم لمطابقة النمط في النصوص.
%	نسبة مئوية	يُمثل أي عدد من الحروف (بما في ذلك الصفر).
_ (underscore)	شرطة سفلية	يُمثل حرفاً واحداً فقط.
BETWEEN	بين	يُستخدم للتحقق من أن القيمة ضمن نطاق (مثل: SALARY BETWEEN 30000 AND 50000).
ORDER BY	رتب حسب	يُستخدم لترتيب النتائج تصاعدياً أو تنازلياً.
ASC	تصاعدي	الترتيب من الأصغر إلى الأكبر (افتراضي).
DESC	تنازلي	الترتيب من الأكبر إلى الأصغر.

+ العمليات الحسابية (Arithmetic Operators)

الرمز	الترجمة	المعنى
+	زائد	جمع
-	ناقص	طرح
*	نجمة	ضرب
/	شرطة مائلة	قسمة
AS	باسم	يُستخدم لإعطاء اسم مؤقت للعمود الناتج (مثل: SALARY * 1.1 AS NewSalary).

أوامر التعديل (INSERT, UPDATE, DELETE)

الأمر	الترجمة	الشرح
INSERT	إدراج	لإضافة صف جديد إلى الجدول.
VALUES	قيم	تُستخدم بعد INSERT لتحديد القيم المُدخلة.
DELETE	حذف	لحذف صفوف من الجدول.
UPDATE	تحديث	لتعديل قيم في صفوف موجودة.
SET	تعيين	يحدد العمود والقيمة الجديدة في أمر UPDATE.
WHERE	حيث	يحدد الصفوف التي سيتم حذفها أو تحديثها.
BULK LOADING	تحميل جماعي	إدخال كمية كبيرة من البيانات دفعة واحدة.
WITH DATA	مع البيانات	تُستخدم عند إنشاء جدول جديد ونسخ البيانات إليه من جدول آخر.

رموز وعلامات خاصة

الرمز	المعنى
;	يُنهي كل أمر SQL (مثل النقطة في الجملة).
' '	يُستخدم لتحديد النصوص (مثل: 'Ahmed').
" "	أحياناً يُستخدم للنصوص أو الأسماء، لكن غالباً يُستخدم الاقتباس الفردي.
*	نجمة: تعني "كل الأعمدة".
<>	لا يساوي (مثل: SALARY <> 5000).
=	يساوي

الرمز	المعنى
<, >, <=, >=	أصغر، أكبر، أصغر أو يساوي، أكبر أو يساوي
AND, OR, NOT	و، أو، ليس (لربط الشروط المنطقية)

أمثلة تطبيقية من النص

مثال 1:

```
SELECT E.Fname, E.Lname, S.Fname, S.Lname
FROM EMPLOYEE AS E, EMPLOYEE AS S
WHERE E.Super_ssn = S.Ssn;
```

- الترجمة: استرجع الاسم الأول والاسم الأخير لكل موظف، والاسم الأول والاسم الأخير لمشرفه المباشر.
- الشرح: استخدمنا الجدول EMPLOYEE مرتين بأسماء مستعارة: E للموظف، و S للمشرف.

مثال 2:

```
UPDATE EMPLOYEE
SET SALARY = SALARY * 1.1
WHERE DNO IN (SELECT DNUMBER FROM DEPARTMENT WHERE DNAME = 'Research');
```

- الترجمة: عدّل راتب الموظفين في قسم "الأبحاث" ليصبح 110% من راتبهم الحالي.
- الشرح: استخدمنا IN مع استعلام فرعي لتحديد أرقام الأقسام التي اسمها "Research".

مثال 3:

```
INSERT INTO EMPLOYEE VALUES ('John', 'Smith', '123456789', '1965-01-09',
...);
```

- الترجمة: أدخل صفًا جديدًا في جدول الموظفين.
- الشرح: يجب أن تكون القيم بنفس الترتيب الذي تم به تعريف الأعمدة في CREATE TABLE.

ملاحظات مهمة 

- إذا أردت نتائج فريدة DISTINCT لا يزيل التكرارات تلقائياً، لذا استخدم SQL.
 - الجداول في SQL تُعامل كـ مجموعات غير مرتّبة، لذا استخدم ORDER BY للترتيب.
 - يمكن استخدام استعلامات فرعية (Subqueries) داخل WHERE أو FROM .
 - القيمة NULL تعني "غير معروفة" أو "غير محددة"، وليست صفر أو نص فارغ.
-